Enhancing LLM-Based Bug Reproduction for Android Apps via Pre-Assessment of Visual Effects

Xiangyang Xiao, Huaxun Huang*, Rongxin Wu

***Abstract*—In the development and maintenance of Android apps, the quick and accurate reproduction of user-reported bugs is crucial to ensure application quality and improve user satisfaction. However, this process is often time-consuming and complex. Therefore, there is a need for an automated approach that can explore the Application Under Test (AUT) and identify the correct sequence of User Interface (UI) actions required to reproduce a bug, given only a complete bug report. Large Language Models (LLMs) have shown remarkable capabilities in understanding textual and visual semantics, making them a promising tool for planning UI actions. Nevertheless, our study shows that even when using state-of-the-art LLM-based approaches, these methods still struggle to follow detailed bug reproduction instructions and replan based on new information, due to their inability to accurately predict and interpret the visual effects of UI components. To address these limitations, we propose LTGDroid. Our insight is to execute all possible UI actions on the current UI page during exploration, record their corresponding visual effects, and leverage these visual cues to guide the LLM in selecting UI actions that are likely to reproduce the bug. We evaluated LTGDroid, instantiated with GPT-4.1, on a benchmark consisting of 75 bug reports from 45 popular Android apps. The results show that LTGDroid achieves a reproduction success rate of 87.51%, improving over the state-of-the-art baselines by 49.16% and 556.30%, while requiring an average of 20.45 minutes and approximately $0.27 to successfully reproduce a bug. The LTGDroid implementation is publicly available at https://github.com/N3onFlux/LTGDroid.**

***Index Terms*—Automated Bug Reproduction, Large Language Model, Prompt Engineering.**

I. Introduction

ANDROID apps have become an indispensable part of modern life. Debugging and fixing bugs are critical tasks in Android app development. Studies have shown that 88% of users may stop using an app when they experience repeated bugs [1], highlighting the need to quickly resolve bugs in Android apps to maintain user satisfaction. Bug reporting systems, such as GitHub Issue Tracker [2] and Google Code [3], are widely used to collect and manage bug reports. These reports typically describe both the observed and expected behaviors, and often provide steps to reproduce (S2Rs) the bugs, which assist developers in reproducing and resolving the issues. However, these reports are often written in informal natural language, which can be imprecise and incomplete, making it difficult for developers to reproduce the reported bugs. Even with well-documented reports, reproducing bugs can be challenging due to the complex, event-driven UIs of Android apps. These challenges make the bug reproduction process time-consuming and error-prone, reducing its effectiveness. Automated bug reproduction techniques can assist developers in quickly reproducing bugs, thereby improving repair efficiency and software quality.

Various approaches have been proposed for automated bug reproduction [4]–[12]. A class of methods [4]–[7], [9] relies on traditional Natural Language Processing (NLP) techniques to extract steps to reproduce (S2R) from bug reports and map them to user interface (UI) widgets of the target app. Most of these approaches are based on identifying specific grammar patterns or analyzing the linguistic structure of the bug report to extract relevant actions and entities. However, such reliance on predefined or heuristic-based language patterns constrains their ability to understand the full semantic richness and diversity of user-written S2R instructions. Consequently, these approaches may struggle to accurately interpret bug reports whose descriptions fall outside commonly observed patterns, resulting in difficulty in bridging the semantic gap between natural language bug reports and the actual UI context of the app. Recently, Large Language Models (LLMs) have demonstrated capabilities in understanding the semantics of text, code, and images. Some studies [11], [12] have attempted to use LLMs to guide LLM-based agents in generating UI actions by combining contextual information such as the S2R, the current UI screen, and the reproduction history. However, approaches that rely solely on the above contextual information are still limited in their ability to predict the outcomes of UI actions. As a result, incorrect UI actions may be generated, causing the reproduction process to deviate from the correct steps. This deviation is fundamentally due to the inability to predict the consequences of each action. In our ablation study (see Section V-D), compared with the ablated version without the ability to predict the consequences of UI actions, our approach increased the overall reproduction success rate from 51.11% to 87.51%, corresponding to a relative improvement of 67.31%.

To address these challenges, we propose LTGDroid, an LLM-based approach for generating UI actions for Android apps based on bug reports. Unlike existing methods that rely on LLMs to directly plan the next UI action based on screen semantics, LTGDroid enumerates all available UI actions on each UI screen and pre-assesses their resulting runtime behaviors. LTGDroid then guides the LLM to select UI actions that align with the S2R in the bug reports. Although there is a potential concern that enumerating all possible actions may theoretically reduce efficiency in bug reproduction, our experimental results show that this pre-assessment procedure

The authors are with Xiamen University, Xiamen, Fujian, China (e-mail: xiangyangxiao@stu.xmu.edu.cn; huanghuaxun@xmu.edu.cn; wurongxin@xmu.edu.cn). Huaxun Huang is the corresponding author.

only leads to a limited increase in the number of UI actions while improving the overall reproduction success rate.

To evaluate the effectiveness of LTGDroid, we built a benchmark dataset comprising 75 bug reports drawn from 45 open-source Android apps. We compare LTGDroid with state-of-the-art LLM-based approaches, including ReBL [12] and AdbGPT [11], all powered by GPT-4.1 for a fair comparison. The evaluation results show that LTGDroid achieves an overall success rate of 87.51%, outperforming the baselines with relative improvements of 49.16% and 556.30%, respectively. Efficiency analysis further demonstrates that LTGDroid requires an average of 20.45 minutes and approximately $0.27 to successfully reproduce a single bug.

In summary, the main contributions of this paper are as follows:

- To the best of our knowledge, we are the first to investigate leveraging runtime behaviors caused by UI actions to assist LLMs in automated bug reproduction.
- We implemented an open-source tool, LTGDroid, that effectively utilizes runtime behaviors for automated bug reproduction.
- We evaluate LTGDroid on real-world bug reports, demonstrating that it achieves a high success rate of bug reproduction within a reasonable number of UI actions.

II. Background

A. Android UI Model

In Android apps, a UI page can be represented as a tree-structured UI state S, where each node corresponds to a UI widget w. This tree-based representation follows the hierarchical layout mechanism of Android, where a root view contains multiple nested child views. Each widget w is associated with a set of attributes (e.g., `android:text`, `android:id`) that determine both its visual appearance and its interaction semantics.

A UI action is formally defined as $a = (t, w, o)$, where t denotes the action type (e.g., `Click`, `LongClick`, `InputText`, `Swipe`), w represents the target widget on which the action is performed (which may be null, such as in a global `Swipe` action), and o is an optional data field (e.g., the input text in an `InputText` action). Executing an action a on a UI state S results in a transition to a new state S', written as $S \xrightarrow{a} S'$. Since the view hierarchy does not always fully reflect the rendered visual state of an app (e.g., enabling night mode alters the appearance while the corresponding view hierarchy remains unchanged), we treat each action as leading to a new UI state regardless of whether the underlying view hierarchy changes.

Such action-induced transitions collectively form a UI state transition graph of the app, which provides the foundation for automated exploration. In the context of bug reproduction, modeling UI states and actions in this way enables systematic reasoning about how a sequence of user interactions can trigger and reproduce a reported bug.

B. Large Language Models

Large Language Models (LLMs) typically refer to Transformer-based models that contain billions of parameters and are trained on massive text corpora. Representative examples include ChatGPT [13], GPT-4 [14], PaLM [15], and LLaMA [16]. Beyond processing and understanding natural language, modern LLMs are increasingly capable of capturing the semantics of images. Through multimodal training, these models can jointly analyze and interpret textual and visual information, enabling them to extract meaningful signals from images and align them with textual descriptions. This cross-modal semantic understanding allows LLMs to reason about complex relationships between different data modalities. Such capabilities make LLMs a promising foundation for automating bug report reproduction, where accurate interpretation of both textual bug descriptions and visual UI contexts is essential.

Another important advancement is the use of Chain-of-Thought (CoT) reasoning [17]. CoT enables LLMs to explicitly decompose complex tasks into intermediate reasoning steps, rather than producing direct answers in a single step. This mechanism has been shown to significantly improve LLMs' performance on tasks requiring logical reasoning, planning, or multi-step decision making [18], [19]. In the context of bug reproduction, CoT is particularly valuable as it enables LLMs to generate UI actions in an iterative, step-by-step manner rather than producing the entire sequence in a single pass. By reasoning over the current UI state, selecting the next action, and assessing its effect on the app, CoT ensures coherence across multi-step action generation. This structured reasoning process reduces the likelihood of incomplete or inconsistent action sequences, thereby enhancing the accuracy and reliability of LLM-driven bug reproduction.

anlalalu opened on Dec 24, 2019

Describe the bug

crash when cutting a folder and paste in it

To Reproduce

1. create a folder (named 'test2') in an empty folder
2. long click 'test2' folder
3. tap 'cut'
4. click 'test2' folder to go into the folder
5. tap 'paste'

Fig. 1. A real bug report (AmazeFileManager#1796) used as the motivating example.

III. Motivation

We use a real bug report shown in Fig. 1 to illustrate the limitations of existing methods and to motivate our work. The example is drawn from AmazeFileManager [20], a popular file management app on GitHub with over 5.7K stars. In Issue

Fig. 2. Reproduction actions generated by LTGDroid and baseline approaches for the motivating bug report.

AmazeFileManager#1796 [1], the developer raised a bug report: when cutting a folder and pasting it into itself, the app crashes. Our goal is to transform the reproduction steps provided in the bug report (Fig. 1) into the correct UI actions illustrated in Fig. 2 (a)–(e). Specifically, (a) clicks the entry of the empty folder "Alarms", leading to (b), where the floating button at the bottom-right corner is clicked to open the creation menu and the option to create a new folder is selected. This brings up the dialog in (c), where the name "test2" is entered and the "CREATE" button is pressed, resulting in the newly created folder shown in (d). The folder is then long-pressed and cut using the top-right cut button. Finally, in (e), after entering the test2 folder, the paste button in the top-right corner is clicked, thereby completing all the UI actions required to reproduce the bug.

We apply the following two baseline approaches to the real bug report shown in Fig. 1:

- **AdbGPT [11]**: The first approach that leverages the LLM for bug report reproduction. Specifically, AdbGPT identifies the S2R entities from the bug report and then uses these extracted entities as prompts, together with the available UI widgets, to determine the most suitable UI actions for replaying the S2R.
- **ReBL [12]**: The current state-of-the-art method for bug report reproduction. ReBL improves upon AdbGPT by eliminating the dependency on S2R extraction. It employs a feedback-driven mechanism that integrates the bug report, the current app state, and the history of reproduction actions to capture a richer UI context. This design enables ReBL to more effectively handle incomplete or ambiguous information in bug reports.

Although we provided both AdbGPT and ReBL with the complete S2R and all relevant information from the bug report, neither of them succeeded in reproducing the bug. The "Baselines" part in Fig. 2 shows the UI actions generated by AdbGPT and ReBL when attempting to reproduce the step "create a new folder in an empty directory." As shown, both methods overlooked the critical clue of the "empty folder". For example, when resolving ambiguous S2R, ReBL attempts to optimize AdbGPT by leveraging non-S2R information, including contextual details from the bug report itself, as well as the app's current UI state and reproduction history. In the interface shown in Fig. 2(f), ReBL considers clicking the floating button at the bottom-right corner as the best option for creating a new folder, since this button might open a menu with such functionality. Furthermore, when exploring the interface in Fig. 2(j), ReBL tries to find a paste button to perform Step 5 from the bug report in Fig. 1. However, since the paste button does not exist in this state, ReBL continues searching for alternative UI widgets, which causes the triggered actions to increasingly diverge from the intended reproduction path, eventually making the process unable to terminate in a timely manner.

To overcome these limitations, LTGDroid does not rely solely on the current UI state and reproduction history when selecting UI actions. Instead, LTGDroid enumerates all possible UI actions available on the current screen and executes them to observe their visual runtime behaviors. The extracted visual effects are then provided to the LLM, which

[1]https://github.com/TeamAmaze/AmazeFileManager/issues/1796

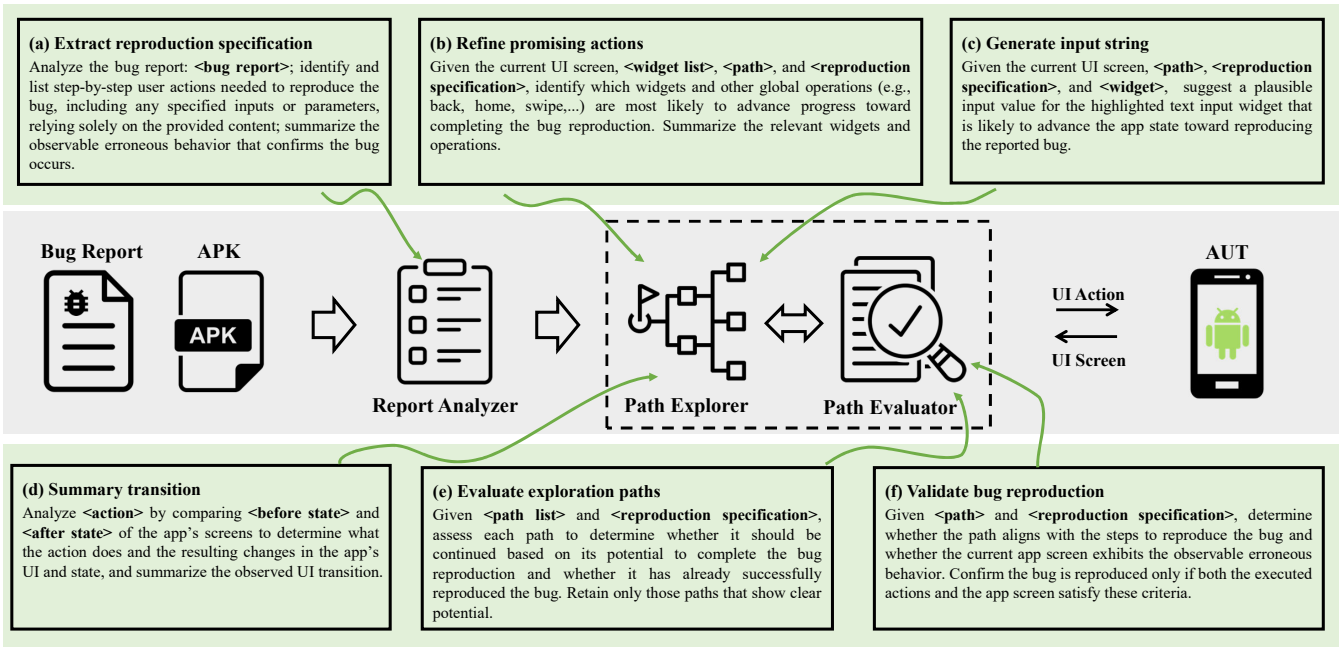

Fig. 3. Overview of the workflow and prompt structures employed in LTGDroid for automated bug reproduction.

uses them to select the action most likely to advance the bug reproduction task. For example, when LTGDroid reaches the interface shown in Fig. 2(a), it proactively attempts all possible UI actions on that screen (including clicking the empty folder "Alarms") to capture the corresponding visual runtime behaviors. Based on these observations, LTGDroid can determine that entering the empty folder first (Fig. 2(a)) and then clicking the floating button (Fig. 2(b)) is more likely to accomplish the intended step. LTGDroid then continues along this path until it reaches Fig. 2(e), where the UI action successfully triggers the bug. Although this exploration incurs additional overhead to collect the visual effects of different UI actions, our experimental results demonstrate that such an investment significantly improves LTGDroid's success rate in bug reproduction (see Section V-D).

IV. Approach

We propose LTGDroid, an automated bug reproduction technique for Android apps. Specifically, LTGDroid takes a bug report and the app under test (AUT) as input, and then automatically generates and executes a sequence of UI actions that trigger the bug described in the report. Essentially, the process of bug reproduction can be abstracted as starting from the initial state of the AUT and progressively exploring feasible paths of UI actions until reaching a bug whose behavior is consistent with the description in the bug report. We therefore model bug reproduction as a state-transition process that begins with an empty initial graph and dynamically expands during exploration.

Building on this idea, LTGDroid is designed with three core modules: Report Analyzer, Path Explorer, and Path Evaluator, as illustrated in Fig. 3. First, the Report Analyzer parses the complete natural language bug report to extract the bug reproduction specification, including the steps to reproduce (S2R) and observable error symptoms, which serve as the goals for subsequent exploration and evaluation. Then, the Path Explorer and Path Evaluator iteratively generate and filter candidate UI actions based on the UI screenshots and the corresponding view hierarchy returned by the AUT, progressively exploring potential interaction paths until the Path Evaluator confirms that a path has successfully reproduced the bug.

In addition to the overall workflow, Fig. 3 also illustrates the structures of the prompts employed in each module. These prompt structures make use of a set of attributes (e.g., **<bug report>**, **<widget list>**) to represent placeholders within the prompts. The detailed descriptions of these attributes are summarized in Table I. Due to space limitations, the figure and table only present the description of the prompts rather than their complete content. The full prompt details are publicly available in our GitHub repository [21].

A. Report Analyzer

Our goal is to achieve automated bug reproduction on a given AUT using the original bug report as input. However, the structure and content of bug reports are often highly variable: they may include S2R, observed failure symptoms, error logs, or even potential root-cause analyses. The completeness and level of detail largely depend on the reporter's expertise. Relying solely on the S2R section may omit crucial information [12], while directly using the original bug report as the prompt introduces excessive noise, which not only interferes with LLMs' reasoning but also significantly increases token consumption.

To address this challenge, we adopt a compromise strategy. Instead of using only the S2R section or the original report,

we leverage LLMs to analyze and interpret the complete bug report, distilling and complementing the S2R while also summarizing the observable error symptoms at the UI level. By leveraging the natural language understanding and reasoning capabilities of LLMs, the Report Analyzer preserves key information while filtering out redundant details, thus providing clearer and more reliable reproduction targets for the Path Explorer and Path Evaluator. Specifically, we design a structured prompting strategy (see Fig. 3(a)) that enables the extraction of a precise exploration objective from the bug report.

TABLE I
PROMPT ATTRIBUTE DESCRIPTIONS IN LTGDROID

Attribute	Description
bug report	The original bug report, including the title, body, and comments in plain text.
widget	The XML-formatted representation of a widget, derived from its view hierarchy properties to capture its semantic meaning in natural language. **Example:** <EditText android:text="Enter Name"/>.
widget list	The list of widget elements in the current view hierarchy, each paired with its executable UI actions.
path	The currently explored path, represented by all transitions along the path and reflecting the exploration history of the corresponding STG node; **Example:** (1) Clicking "Alarms" opens the folder, updates the view to show its empty contents; (2) Clicking the floating action button expands a mini-menu with quick actions for creating a folder, file, or cloud connection.
path list	The list of all ongoing exploration paths in the current iteration, where each element follows the same structure as <path>.
reproduction specification	The S2R and observable error symptoms distilled by the Report Analyzer from the bug report, representing the exploration objective.
action	The natural language description of a UI action; **Example:** input "test2" in widget <EditText android:text="Enter Name"/>.
before state/ after state	The UI screen before/after executing an action, consisting of the screenshot and the name of the current activity.

B. Path Explorer

In LTGDroid, we introduce a pre-exploration process that systematically collects and analyzes all possible UI actions and their corresponding state transitions based on the current UI state of the AUT. The goal of this process is not to directly complete bug reproduction but to generate a set of more comprehensive candidate exploration paths, which will later serve as input for the Path Evaluator. By performing a preliminary assessment of UI actions, LTGDroid provides a more targeted foundation for exploration, thereby enabling the Path Evaluator to retain only those paths most likely to trigger the target bug.

Specifically, the Path Explorer enumerates all executable UI actions from the current UI state and analyzes the effect of each action and the resulting visual changes in the app's UI and state, which facilitates subsequent filtering in the Path Evaluator. To enumerate all available UI actions, LTGDroid first parses the current XML-based view hierarchy to construct a widget tree and inspects each widget's attributes (e.g., `android:clickable`). Based on these attributes and widget types, it systematically derives a comprehensive set of feasible UI actions. In total, LTGDroid supports six major categories of actions: tapping or long-pressing a UI widget (`Click`, `LongClick`); performing directional swipe gestures (`Swipe`); entering LLM-generated text (`InputText`; generated using the prompt shown in Fig. 3(c)); switching between portrait and landscape orientation (`Rotate`); and invoking system-level key events (`Press`; e.g., *Back*, *Enter*, *Delete*, *Home*).

A straightforward approach would be to evaluate every possible action in the current state. However, this method faces two major challenges in practice: (1) A large portion of UI actions and their visual effects are irrelevant to the exploration objective, leading to inefficiency in path evaluation and potentially exceeding the LLM's maximum token limit. (2) The number of actions in a single UI state can be substantial, and evaluating them one by one would incur prohibitive time costs and token consumption.

To address these issues, LTGDroid invokes the LLM to analyze the semantic information of the current screen and filter out actions unrelated to the exploration objective. This reduces the input size for the action planner while preserving relevance. In practice, we design a structured prompt (see Fig. 3(b)) that combines the UI screen, the exploration objective (i.e., the <reproduction specification>), and the exploration path context to refine the action set. The process is detailed in Algorithm 1. In line 2, LTGDroid first extracts all executable actions from the current UI state (denoted as $actions$). LTGDroid then invokes the LLM to discard actions that are unlikely to contribute to the objective (lines 3).

After refining the action set, LTGDroid executes each remaining action and captures the UI screens before and after execution. These, together with contextual information of the action, are summarized as UI transitions and added to the $transitions$ collection for subsequent use by the Path Evaluator (lines 4–11). To maintain independence among actions, LTGDroid rolls back the emulator to a previously saved snapshot after each execution (line 10). For example, in the bug reproduction task shown in Fig. 1, LTGDroid first extracted all executable actions in the current UI state (Fig. 2(a)). LTGDroid then employed the LLM to filter out irrelevant ones. The retained candidates included actions such as "clicking the floating button in the bottom-right corner" and "clicking the entry for the Alarm folder". LTGDroid then executed these actions iteratively, recorded the corresponding UI transitions, and added them to the interaction history of the corresponding exploration path for subsequent evaluation.

In practice, we leverage an LLM to summarize the UI transitions by designing a structured prompt (see Fig. 3(d)). Importantly, we focus on observable UI changes rather than

Algorithm 1: Pre-Exploration of UI Actions

Input: $state$: Current UI state of the AUT, $objective$: Exploration Objective
Output: $transitions$: Set of candidate UI transitions

$transitions \leftarrow \emptyset$;
$actions \leftarrow$ extractActions($state$);
$actions \leftarrow$ filterActions($actions, objective$);
foreach $action \in actions$ **do**
  $screen\_before \leftarrow$ captureScreen();
  execute($action$);
  $screen\_after \leftarrow$ captureScreen();
  $transition \leftarrow$ summTrans($action, screen\_before, screen\_after$);
  $transitions \leftarrow transitions \cup \{transition\}$;
  rollback($state$);
end
return $transitions$;

system-level metrics (e.g., memory usage or CPU load), as our goal is to capture semantically meaningful state evolutions from the user's perspective. To this end, we instruct the LLM to analyze the captured UI screenshots together with the action-related contextual information and generate a structured summary at multiple levels of granularity. Specifically, the LLM describes both what the action does and the resulting changes in the app's UI and state, with emphasis on phenomena such as widget additions or removals, visibility updates, content modifications, layout shifts, and screen or Activity switches. This structured characterization ensures that the extracted UI transitions are precise, consistent, and directly aligned with the needs of the subsequent path evaluation process.

Notably, although the LLM may occasionally filter out actions that are actually relevant to the exploration objective, our experiments indicate that such cases only rarely affect LTGDroid's overall effectiveness (see Reason #1 in Section V-C).

C. Path Evaluator

Based on the exploration results provided by the Path Explorer, the Path Evaluator filters the ongoing exploration paths, pruning branches that deviate from the exploration objective and retaining only the most promising ones for further expansion in the next iteration. This yields a BFS-like bounded exploration process that continues until the evaluator determines that a path has successfully fulfilled the exploration objective. For illustration, consider the real bug reproduction task illustrated in Fig. 1. LTGDroid first obtains UI transitions relevant to the exploration objective through pre-assessment (see Section IV-B) in the current UI state (Fig. 2(a)). The evaluator then analyzes these paths: the path leading from (a) to (b), entering an empty folder, is considered beneficial for reaching the objective and thus retained for further iteration, whereas the path from (a) directly clicking the floating button to (g), entering a menu, is deemed irrelevant and is pruned. This iterative exploration and evaluation continues until the exploration path from (a) to (e) is confirmed to reproduce the bug, at which point the process ends.

Specifically, we design a structured prompt (see Fig. 3(e)) that combines the exploration objective (i.e., the <reproduction specification>) with the ongoing exploration paths to assess whether a path is worth further exploration and whether it has already triggered the target bug. The evaluator then retains the most promising paths and forwards them to the Path Explorer for the next round of expansion.

To mitigate the risk of the LLM prematurely determining that an exploration path has reached the objective due to inherent randomness or limited contextual information, we incorporate an additional verification step before concluding the process. Specifically, upon the evaluator's initial identification of a path as satisfying the exploration objective, we validate this outcome by employing a dedicated prompt (see Fig. 3(f)) to examine the corresponding UI screen and ensure that the observed behavior aligns with the expected bug manifestation. For crash-related bugs, the evaluator monitors error logs to confirm the occurrence of the crash. This additional verification reduces the chance of inaccurate conclusions caused by LLM limitations and helps ensure that identified paths correspond authentically to the target bug reproduction.

V. Evaluation

To evaluate LTGDroid, we formulate three key research questions:

- **RQ1:** How effective and efficient is LTGDroid in reproducing bug reports?
- **RQ2:** How do the modules of LTGDroid affect its effectiveness and efficiency?
- **RQ3:** How does LTGDroid perform compared to three baseline methods in terms of effectiveness and efficiency?

A. Dataset

In this study, we constructed a dataset for Android bug reproduction. Specifically, we first integrated the evaluation datasets used in four existing studies: ReCDroid [5], ReproBot [7], AdbGPT [11], and ReBL [12], together with two manually reproduced bug report datasets, AndroR2 [22] and Themis [23]. During the data cleaning process, we removed duplicate entries and reports not hosted on GitHub (since our bug reports were collected via the GitHub REST API [24]), and further filtered out unusable samples through manual reproduction. Unusable samples included reports with broken links, missing corresponding APK files, dependencies on special initialization resources (e.g., account login, importing audio/video or SMS records). After filtering, 48 valid samples remained, including 37 crash reports and 11 non-crash reports. Since many reports in the above datasets corresponded to older defects, we additionally collected 27 new bug reports submitted in 2024 or later from some of the apps in the existing datasets as well as several popular GitHub open-source apps (Stars $>$ 200). These included 14 crash reports and 13 non-crash reports.

TABLE II
APP SUBJECTS USED IN OUR EVALUATION (K=1,000, M=1,000,000)

App	Feature Description	#Google Play Installations	#Github Stars	#Lines of Code
ActivityDiary	Activity Scheduler	1K+	75	18.2K
AmazeFileManager	Lightweight File Manager	1M+	5.7K	114.8K
AndrOBD	OBD Car Diagnostics	1K+	1.7K	61.9K
android-noad-music-player	Local Music Player	10K+	69	13.4K
android-obd-reader	OBD Car Diagnostics	-	827	28.1K
anglers-log	Fishing Log & Analytics	100K+	31	134.2K
Anki-Android	Spaced Repetition Flashcards	10M+	9.9K	282K
AntennaPod	Podcast Manager	1M+	7.2K	107.8K
AnyMemo	Spaced Repetition Flashcards	100K+	153	33.4K
APhotoManager	Local Photo Gallery	-	231	48.2K
AsciiCam	Real-Time ASCII Camera	100K+	134	3.3K
Birthdroid	Birthday Reminder	-	14	2.9K
FastAdapter	RecyclerView Adapter	-	3.9K	15K
fastnfitness	Fitness Tracker	5K+	302	36.2K
FirefoxLite	Web Browser	5M+	292	146.6K
focus-android	Web Browser	10M+	2.1K	81.2K
gnucash-android	Personal Finance Manager	10K+	1.2K	53.3K
gpstest	GPS Test & Diagnostics	1M+	2K	26.8K
jtxBoard	Journal, Notes Manager	10K+	499	296K
KISS	Minimalist Launcher	100K+	3.2K	44.2K
kiwix-android	Offline Wikipedia Reader	1M+	1.1K	80.6K
LibreNews-Android	News Aggregator	-	36	2.5K
LocalMaterialNotes	Local Markdown Notes	1K+	239	21.5K
markor	Markdown & To-Do Notes	-	4.6K	65.7K
MaterialFBook	Facebook Client	-	139	9.4K
MaterialFiles	File Manager	1M+	7.3K	83.7K
Memento-Calendar	Calendar & Event Manager	10K+	210	31K
mhabit	Habit Tracker & Goal Setter	-	767	76.4K
microMathematics	Scientific Calculator	50K+	612	80.6K
NewPipe	Lightweight YouTube Client	-	34.8K	169.3K
NewsBlur	RSS Reader	50K+	7.1K	418.3K
NotallyX	Minimalistic Note Manager	1K+	334	309.6K
Omni-Notes	Note Manager	100K+	2.8K	46.4K
OpenCalc	Scientific Calculator	100K+	1.3K	16K
osmdroid	OpenStreetMap Tools	10K+	3K	132.4K
privacy-friendly-todo-list	To-Do List	-	110	16.4K
privacy-friendly-weather	Weather	-	121	226.9K
qksms	Customizable SMS Manager	1M+	4.6K	59.5K
screenrecorder	Screen Recording	50K+	122	7.2K
thunderbird-android	Email Client	5M+	12.4K	314.1K
Transportr	Public Transport Tracker	10K+	1.1K	40.5K
trickytripper	Trip Planner	5K+	47	3.5M
uhabits	Habit Tracker & Goal Setter	5M+	9K	53.3K
URLCheck	Batch URL Checker	100K+	1.5K	23.7K
WiFiAnalyzer	Wi-Fi Analyzer	10M+	4.1K	25K

We constructed a dataset of 75 bug reports from 45 different apps (see Table II), including 51 crash reports and 24 non-crash reports. For each report, we manually annotated the number of ground-truth UI actions required for successful reproduction. Detailed information for all bug reports is provided in Table III. The distribution of these actions is broad, indicating no bias toward overly simple or complex bugs: 28 reports required 1–5 actions, 34 reports 6–10 actions, 11 reports 11–15 actions, and 2 reports more than 15 actions. Notably, all bug reports in our dataset are kept in plain-text form to ensure compatibility with existing evaluation baselines. If needed, LTGDroid can be easily extended to support image-based bug reports.

B. Experimental Setup

Our experiments were conducted on a Mac mini equipped with an Apple M4 chip (10-core CPU) and 32 GB of memory. The experimental environment adopted the official Android emulator configured with an Android 9.0 ARM system image, 4 CPU cores, 4 GB of memory, and hardware acceleration enabled.

It is worth noting that, throughout the evaluation, we aimed to faithfully replicate the real-world scenarios faced by developers when reproducing bug reports. To this end, the experiments only automated the installation and launch of applications without relying on any manually crafted initialization scripts. Furthermore, to ensure fairness, all components involving LLMs uniformly employed GPT-4.1, a multimodal model developed by OpenAI.

TABLE III
BUG REPORTS USED IN OUR EVALUATION

ID	Report	Type	#GA
1	ActivityDiary#118	Crash	15
2	ActivityDiary#285	Crash	7
3	AmazeFileManager#1796	Crash	10
4	AmazeFileManager#4185	Crash	4
5	AndrOBD#144	Crash	6
6	android-noad-music-player#1	Crash	2
7	android-obd-reader#22	Crash	7
8	Anki-Android#10584	Crash	10
9	Anki-Android#4586	Crash	6
10	Anki-Android#4707	Crash	6
11	Anki-Android#4977	Crash	3
12	Anki-Android#5638	Crash	4
13	Anki-Android#6432	Crash	32
14	AnyMemo#422	Crash	4
15	AnyMemo#440	Crash	7
16	APhotoManager#116	Crash	2
17	AsciiCam#17	Crash	3
18	Birthdroid#13	Crash	6
19	FastAdapter#394	Crash	1
20	fastnfitness#142	Crash	9
21	fastnfitness#293	Crash	15
22	FirefoxLite#5085	Crash	5
23	gnucash-android#664	Crash	15
24	jtxBoard#1727	Crash	5
25	kiwix-android#990	Crash	7
26	LibreNews-Android#22	Crash	6
27	LibreNews-Android#23	Crash	7
28	LocalMaterialNotes#385	Crash	9
29	markor#194	Crash	8
30	MaterialFBook#224	Crash	1
31	Memento-Calendar#169	Crash	9
32	microMathematics#39	Crash	4
33	NewPipe#11914	Crash	8
34	NewPipe#11915	Crash	10
35	NewsBlur#1053	Crash	4
36	NotallyX#517	Crash	6
37	NotallyX#569	Crash	3
38	NotallyX#609	Crash	4
39	Omni-Notes#377	Crash	2
40	Omni-Notes#745	Crash	10
41	OpenCalc#493	Crash	2
42	OpenCalc#532	Crash	2
43	osmdroid#1030	Crash	9
44	privacy-friendly-weather#61	Crash	4
45	qksms#585	Crash	6
46	screenrecorder#25	Crash	8
47	thunderbird-android#3255	Crash	4
48	Transportr#972	Crash	5
49	trickytripper#42	Crash	11
50	uhabits#1545	Crash	10
51	URLCheck#493	Crash	11
52	anglers-log#151	Non-crash	11
53	anglers-log#347	Non-crash	8
54	Anki-Android#5753	Non-crash	7
55	AntennaPod#7611	Non-crash	3
56	AntennaPod#7866	Non-crash	5
57	focus-android#3297	Non-crash	13
58	gpstest#404	Non-crash	12
59	KISS#1481	Non-crash	11
60	LocalMaterialNotes#327	Non-crash	7
61	LocalMaterialNotes#415	Non-crash	19
62	LocalMaterialNotes#416	Non-crash	4
63	markor#1020	Non-crash	15
64	markor#331	Non-crash	8
65	MaterialFiles#1281	Non-crash	8
66	MaterialFiles#1392	Non-crash	5
67	Memento-Calendar#7	Non-crash	5
68	mhabit#270	Non-crash	9
69	OpenCalc#448	Non-crash	6
70	OpenCalc#510	Non-crash	8
71	OpenCalc#524	Non-crash	5
72	privacy-friendly-todo-list#158	Non-crash	11
73	thunderbird-android#3971	Non-crash	6
74	uhabits#2112	Non-crash	9
75	WiFiAnalyzer#222	Non-crash	4

1) Evaluation Metrics: We evaluated all tools from two perspectives: *effectiveness* and *efficiency*.

Effectiveness was assessed by the success rate (**SR**) formulated as follows:

$$\text{SR} = \frac{\text{\# of Successfully Reproduced Bug Reports}}{\text{\# of Bug Reports in Dataset}} \quad (1)$$

Efficiency was assessed using four metrics: the average number of UI actions (**Actions**), execution time (**Time**), LLM token consumption (**Tokens**), and the corresponding monetary cost (**Cost**). We took an upper bound of 100 UI actions and a maximum execution time of 60 minutes. Any attempt that exceeded either threshold was treated as a failure. Notably, the most complex bug in our dataset requires only 32 UI actions to reproduce manually, which is far below our predefined limit.

To ensure the reliability of our evaluation, three authors of this paper, each with at least two years of Android development experience, independently reviewed the execution results to determine whether the crashes or non-crash errors described in the bug reports were successfully reproduced. Each execution result was independently examined by two authors, and when their assessments differed, the third author re-evaluated the case to resolve the inconsistency. To further mitigate randomness, each experiment was executed three times, and we report the average results across all runs for all metrics.

2) Setup for RQ1: To evaluate the effectiveness and efficiency of LTGDroid, we conducted experiments on the dataset of 75 real-world bug reports. We further analyzed all failure cases to understand the underlying causes and identify potential areas for improvement.

3) Setup for RQ2: To assess the contribution of each module in LTGDroid, we conducted an ablation study by comparing the full system with four variants, each disabling one core module. **LTGDroid w/o RA** removes the bug report analyzer, preventing the system from extracting structured reproduction specifications from the original bug report. **LTGDroid w/o AE** disables UI action enumeration, preventing the system from pre-assessing the effects of multiple UI actions on each screen. **LTGDroid w/o TA** excludes UI transition assessment, preventing the system from analyzing and recording the effects of UI actions. **LTGDroid w/o VB** removes bug behavior verification, preventing the system from confirming whether a candidate execution has triggered the target bug. By comparing these variants against the full LTGDroid system, we quantify how each module contributes to the effectiveness and efficiency.

4) Setup for RQ3: We selected three state-of-the-art LLM-based automated bug reproduction methods as baselines: **AdbGPT** [11], **ReBL** [12], and a visual variant of ReBL (**ReBL-visual**) that incorporates UI screenshots as additional visual context. Notably, since ReBL is already an improvement over AdbGPT, we did not construct a separate AdbGPT-visual variant. Following the same experimental setup as in previous studies, we set an upper limit of 100 UI actions for all methods to ensure a fair performance evaluation, and analyzed their reproduction effectiveness and efficiency within a reasonable action budget. All results were manually verified to ensure the reliability of the comparative analysis.

It is worth noting that methods such as Yakusu [4], ReCDroid [5], ReproBot [7], ScopeDroid [8], and Roam [9], which are not LLM-based, rely on extracting S2R entities from bug reports and matching them to corresponding UI actions. However, most bug reports do not document the initialization steps required when launching a freshly installed app (e.g., permission grants, onboarding screens). Therefore, these methods typically require manually crafted initialization scripts to operate in practice. Moreover, they generally focus only on crash-related bugs and lack support for non-crash bugs. For these reasons, we did not include them as direct comparison baselines in this study.

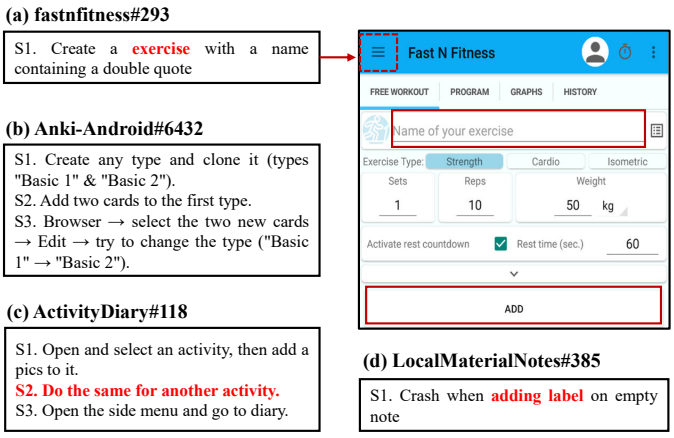

Fig. 4. Representative examples of LTGDroid's bug reproduction failures.

C. Results for RQ1

As shown in Table IV, in terms of effectiveness, LTGDroid achieves a bug reproduction success rate of 88.82% on crash cases and 84.71% on non-crash cases, resulting in an overall success rate of 87.51%. In terms of efficiency, for each successfully reproduced bug, LTGDroid required an average of 27.48 UI actions, 67.07K tokens, $0.27, and 20.45 minutes to complete the reproduction process. These results indicate that LTGDroid is both robust and generalizable, achieving a high bug reproduction success rate with reasonable cost. We further analyzed the 10 reproduced bug reports that LTGDroid failed to reproduce, including 6 crash reports and 4 non-crash reports. We identified the following four main causes.

Reason #1: Incorrectly filtering out relevant UI actions. LTGDroid relies on the LLM to filter out actions deemed irrelevant to the exploration goal, avoiding excessive attempts during the pre-assessment phase. However, due to the LLM's limited understanding of the app and bug report, it may mistakenly exclude critical actions. For example, in *fastnfitness#293* (Fig. 4(a)), the LLM excluded the correct UI action, which involved clicking the top-left menu button leading to the exercise page, while retaining actions that appeared reasonable but were actually incorrect, such as entering and creating exercise records. We found only 1 such case in our evaluation.

Reason #2: Insufficient awareness of reproduction progress. LTGDroid sometimes struggles to accurately track the status of each reproduction step. First, when a reproduction involves many steps, the LLM may misalign the historical context with the reproduction steps, leading to confusion about progress and repeated execution of completed steps. For instance, in *Anki-Android#6432* (Fig. 4(b)), the LLM misinterpreted previously completed steps due to the large number of steps in the S2R, causing repeated actions. Second, when step descriptions are unclear, the LLM may have difficulty determining whether a step has been fully completed. This issue is illustrated by *ActivityDiary#118* (Fig. 4(c)), where S2 required repeating the S1 actions on another activity, but the LLM prematurely proceeded to S3 before fully completing S2. We found 6 such cases in our evaluation.

Reason #3: Inability to infer and complete missing initialization steps. When critical initialization steps are absent

from the bug report, LTGDroid may fail to reproduce the bug. For example, in *LocalMaterialNotes#385* (Fig. 4(d)), the bug report did not mention the need to create at least one label, so LTGDroid did not realize it had to perform this step, resulting in reproduction failure. We found 1 such case in our evaluation.

Reason #4: Limitations of the testing framework. We found 2 bugs related to special operations that are not yet supported by LTGDroid. For instance, *Memento-Calendar#169* requires a swipe within a specific coordinate range, which LTGDroid cannot perform. Similarly, *trickytripper#42* requires a long-click on a UI element marked as non-longclickable, but LTGDroid strictly follows the view hierarchy properties and thus cannot execute this action.

Overall, the reproduction failures primarily stem from the LLM's limitations in understanding and reasoning, as well as the testing framework's operational constraints. These challenges highlight difficulties in handling complex or incomplete bug reports and suggest directions for future improvements in reasoning capability and test framework support.

Answer of RQ1: LTGDroid achieves an overall success rate of 87.51%. For each successfully reproduced bug, it requires an average of 27.48 UI actions, 67.07K tokens, $0.27, and 20.45 minutes. Overall, the results demonstrate that LTGDroid is robust and generalizable while maintaining reasonable cost.

D. Results for RQ2

Table IV presents the comparison among LTGDroid and its variants. First, removing any critical component leads to a noticeable decrease in the reproduction success rate, among which disabling the UI transition assessment module (**LTGDroid w/o TA**) causes the largest drop. Its overall SR is only 51.11%, as the absence of the module prevents the system from predicting the consequences of UI actions during reproduction. This variant also consumes the fewest tokens, mainly because it can only reproduce relatively simple bugs.

Second, when the bug report analyzer is removed (**LTGDroid w/o RA**), the system struggles to reliably extract structured reproduction specifications from raw bug reports, resulting in an overall SR of 62.67%. Both token consumption and execution time of this variant increase significantly because unstructured and noisy reports take up a large amount of the available context budget, thereby reducing the efficiency of action planning.

Third, removing the bug behavior verification module (**LTGDroid w/o BV**) decreases the overall SR to 63.07%, as the system can no longer reliably verify whether the target bug has been triggered, leading to incorrect judgments and premature termination of the reproduction process.

Fourth, removing the UI action enumeration module (**LTGDroid w/o AE**) results in an overall success rate of 68.44%, indicating that the absence of UI action enumeration reduces the effectiveness of UI action evaluation during the reproduction process.

Notably, this variant achieves substantially lower execution time, as disabling action enumeration avoids the pre-assessment of multiple UI actions, which relies on emulator snapshots for state restoration and is time-consuming. In summary, each functional module in LTGDroid is essential not only for ensuring a high success rate but also for balancing efficiency factors, together forming an optimal automated bug reproduction process.

Answer of RQ2: Each functional module in LTGDroid is essential for effective and efficient automated bug reproduction, as removing any component degrades success rate or efficiency. Notably, the UI transition assessment module is the most critical, guiding the interaction flow toward successful reproduction.

E. Results for RQ3

Table IV presents the comparison results between LTGDroid and the baseline methods. Evaluated on 75 bug reports, including 51 crash reports and 24 non-crash reports, LTGDroid outperformed state-of-the-art LLM-based methods (**AdbGPT** [11], **ReBL** [12], and **ReBL-visual**) in terms of effectiveness. Its overall reproduction success rates increased by 556.30%, 49.16%, and 34.85% respectively, and achieved the best performance for both crash and non-crash bug reports.

Notably, the bugs successfully reproduced by LTGDroid tend to involve higher interaction complexity, as reflected by a greater number of ground-truth UI actions required for reproduction (6.40 on average), compared to AdbGPT (3.50), ReBL (5.77), and ReBL-visual (6.29). The successful reproduction traces reported by LTGDroid contain an average of 6.89 UI actions per bug, which is close to the ground-truth average of 6.40, indicating that LTGDroid can report realistic and concise reproduction paths. For the other baselines, the length of a reproduction trace directly corresponds to the number of executed UI actions. As a result, the reproduction traces they produce exhibit much larger deviations from the ground-truth action counts, which incurs additional manual verification effort for developers.

The relatively low success rate of AdbGPT can be attributed to its reliance on extracting S2R entities from bug reports and strictly following these S2R entities step by step. However, most bug reports omit necessary initialization steps (e.g., permissions, onboarding), and since our experiments did not provide manual scripts to handle them, AdbGPT exhibited limited effectiveness. In contrast, LTGDroid, ReBL, and their variants do not depend on S2R extraction. Instead, they plan UI actions directly based on the entire bug report and contextual information such as exploration history. This allows the LLM to reason about and generate the necessary UI actions to handle simple initialization steps that may be missing from the report.

From Table IV, we observe that ReBL-visual achieves higher success rates on both crash and non-crash bugs compared to its original version ReBL. By analyzing the additional cases successfully reproduced by ReBL-visual, we identify two main reasons. First, for some apps, the view

TABLE IV
EVALUATION RESULTS OF LTGDROID AND BASELINES.

Method	Crash SR (%)	Non-crash SR (%)	Overall SR (%)	Actions (#)	Tokens (K)	Cost ($)	Time (m)
AdbGPT	11.76	16.67	13.33	4.10	25.16	0.05	1.40
ReBL	67.98	38.88	58.67	10.98	46.36	0.10	9.13
ReBL-visual	70.59	52.79	64.89	9.39	53.95	0.11	10.09
LTGDroid w/o RA	71.90	43.04	62.67	21.04	65.80	0.25	21.84
LTGDroid w/o AE	75.16	54.17	68.44	9.18	46.16	0.18	7.96
LTGDroid w/o TA	58.82	34.71	51.11	25.74	31.31	0.14	18.40
LTGDroid w/o BV	71.18	45.83	63.07	20.13	56.89	0.23	18.62
LTGDroid	88.82	84.71	87.51	27.48	67.07	0.27	20.45

hierarchy alone does not provide sufficient semantic cues to infer the correct UI actions, particularly when developers neglect accessibility design. In such scenarios, the visual modality introduced by UI screenshots enables ReBL-visual to make more accurate predictions. Second, non-crash bug reports often describe error symptoms in terms of visual manifestations. Since these descriptions are difficult to align with the view hierarchy, ReBL struggles to determine whether the reproduction is successful, whereas ReBL-visual benefits from the additional visual context. These findings highlight the necessity of adopting multimodal models and incorporating multimodal information for more robust bug reproduction.

We further analyze the performance differences between crash and non-crash bugs. Overall, most methods perform worse on non-crash bugs than on crash bugs, with AdbGPT and ReBL showing particularly limited capability in reproducing non-crash bugs. Based on our observations from the non-crash bug reports, reproducing non-crash bugs often appears to be more challenging. This may be because they generally require more UI interactions to trigger, and correctly identifying each necessary action can be more difficult. This observation aligns well with our experimental results, as LLMs require sufficient contextual information to infer the correct reproduction UI actions. Specifically, LTGDroid leverages visual information along with a pre-assessment mechanism to provide the LLM with the most comprehensive contextual information; ReBL-visual provides only visual information of the UI but lacks pre-assessment context; ReBL lacks UI visual information compared to ReBL-visual; and AdbGPT relies on the least information, missing both non-S2R information from the bug reports and feedback information. Consequently, LTGDroid achieves a significantly higher success rate on non-crash bugs, further illustrating the effectiveness of its design in enhancing the overall bug reproduction capability.

In terms of efficiency, LTGDroid required more resource consumption. This can be explained by two main factors. First, LTGDroid is able to successfully reproduce bugs with the highest interaction complexity, and reproducing such complex bugs inherently requires more resource consumption. Second, LTGDroid employs a pre-assessment mechanism that actively explores more potentially useful UI actions for reproduction, which introduces additional overhead. In future work, we plan to improve the efficiency of LTGDroid by reducing LLM token consumption and accelerating the exploration process.

Answer of RQ3: LTGDroid achieves higher bug reproduction success rates than the baseline methods (AdbGPT, ReBL, and ReBL-visual) across both crash and non-crash reports, and can effectively handle complex bugs involving more UI interactions. Although its exploration strategies introduce additional resource overhead, they provide more complete contextual information for the LLM, leading to more accurate and reliable reproduction results. In future work, we plan to further reduce token consumption and accelerate the exploration process to enhance overall efficiency.

VI. THREATS TO VALIDITY

The main threat to the external validity of this study lies in the representativeness of the dataset used for evaluation. To reduce this risk, about two-thirds of the bug reports were collected from prior related studies to ensure comparability with existing work. In addition, considering that the datasets used in those studies may suffer from obsolescence, we further collected bug reports released after 2024 from the applications included in previous studies as well as several popular open-source apps, thereby improving the coverage and realism of our dataset.

Internal validity mainly concerns two aspects. First, evaluating whether a bug is successfully reproduced may introduce human judgment errors. To improve reliability, the three authors conducted a cross-checking procedure in which each execution result was independently examined by two authors, and any disagreement was resolved through discussion with the third author. This collaborative evaluation approach helps reduce individual bias and enhances the objectivity of the assessment results. Second, the inherent randomness of large language models may lead to variations across different experimental runs. To mitigate this issue, each experiment was executed three times, and we report the average results across all runs for all metrics to reduce the impact of random variations.

VII. RELATED WORK

A. Automated Bug Reproduction

Numerous approaches [4]–[9], [11], [12], [25] have been proposed on studying automated bug reproduction. Specifically, early research primarily relied on NLP techniques to

extract S2R from bug reports and match them with UI actions or exploration paths of the application. These methods often integrated program analysis or search strategies to compensate for the incompleteness of information in bug reports. For example, Fazzini et al. proposed Yakusu [4], combining program analysis and NLP to extract reproduction steps from bug reports and map them to UI actions. Zhao et al. proposed ReCDroid [5] and ReCDroid+ [6], which integrate NLP, deep learning, and dynamic GUI exploration to automatically generate event sequences for reproducing reported crashes. Zhang et al. introduced ReproBot [7], which combines NLP analysis with reinforcement learning and leverages search strategies to generate successful reproduction steps. Huang et al. proposed ScopeDroid [8], which builds a state transition graph (STG) based on DroidBot [25] and leverages a multimodal neural model to match bug report steps with UI states and actions, while dynamically expanding the STG to improve reproduction effectiveness. In addition, Zhang et al. proposed Roam [9], which pre-constructs a comprehensive STG and matches potential reproduction paths within it for automated bug reproduction.

With the rise of LLMs, an increasing number of studies have explored leveraging their natural language understanding and reasoning capabilities to assist bug reproduction. Feng et al. proposed AdbGPT [11], which follows the conventional two-stage paradigm of S2R entity extraction and matching, similar to the aforementioned approaches. However, this paradigm has a fundamental limitation: if the S2R description does not provide a complete sequence starting from the initialization step, the reproduction cannot succeed. Our results demonstrate that AdbGPT, constrained by this conventional paradigm, exhibits limited effectiveness. Wang et al. presented ReBL [12], which employs a feedback-driven mechanism that jointly considers bug reports, app states, and reproduction history to guide the LLM in more robust bug reproduction. However, ReBL still relies heavily on the LLM's intrinsic ability to interpret UI semantics, which can sometimes lead to incorrect UI action generation. Our experimental results further demonstrate that LTGDroid achieves higher effectiveness compared with ReBL.

Other studies have approached bug reproduction from videos and crash stack traces. For example, Feng et al. proposed GIFdroid [26], which leverages screen recording and image processing techniques to automate the reproduction of bugs. Huang et al. introduced CrashTranslator [27], which targets stack traces in crash reports and employs the LLM to extract key crash-related information for reproducing crashes in the app. Zhang et al. developed BugSpot [10], which extends the use of LLMs by extracting observable error symptoms from bug reports to assist in identifying target failures during the reproduction process.

B. Large Language Models for Software Engineering

In recent years, the application of large language models in software engineering has been rapidly emerging [28], demonstrating significant advantages across multiple key tasks. For test generation, Zhang et al. investigated how LLMs can automatically synthesize diverse oracle verifiers [29]. Xue et al. proposed the LLM4Fin [30], which integrates LLMs with algorithmic techniques to automatically generate high-coverage test cases from natural language business rules for fintech software acceptance testing. Chen et al. introduced the ChatUniTest [31], which incorporates an adaptive focus-context mechanism and a generate–verify–repair pipeline to improve both accuracy and coverage in LLM-based unit test generation. In GUI testing, Ran et al. proposed the Guardian [32], which enhances the effectiveness of LLM-based feature-driven UI testing by constraining and dynamically correcting UI action planning at runtime. Liu et al. developed the GPTDroid [33], which formulates mobile GUI testing as a question-answering task and introduces functionality-aware memory prompting, enabling LLMs to perform long-term reasoning and interaction to improve test coverage and bug detection. Overall, LLMs in software engineering remain an evolving research frontier. Our work contributes to this line of research by introducing a systematic UI action exploration and pre-assessment mechanism, providing a more robust way to apply LLMs in GUI exploration and bug reproduction.

VIII. Conclusion

In this paper, we proposed LTGDroid, a novel approach that assists LLMs in bug reproduction by enumerating and pre-assessing UI actions. Unlike existing methods that solely rely on UI semantics and contextual history, LTGDroid effectively mitigates errors caused by incorrect actions, thereby improving the overall reproduction success rate. On a benchmark dataset comprising 75 bug reports, LTGDroid achieved a success rate of 85.33%, requiring an average of 27.50 UI actions to complete each task, which demonstrates its advantage of trading moderate efficiency overhead for higher reliability. For future work, we plan to enhance LTGDroid's capability to track exploration progress, improve its ability to recognize manifestations of non-crash bugs, and design more efficient path exploration mechanisms, with the goal of further increasing its practicality and applicability in complex scenarios.

IX. Data Availability

The implementation of LTGDroid, together with the dataset of benchmark bug reports used in our study, are publicly available on the project website https://github.com/N3onFlux/LTGDroid.

REFERENCES

[1] D. Bolton, "88% of people will abandon an app because of bugs," 2025. [Online]. Available: https://www.applause.com/blog/app-abandonment-bug-testing

[2] GitHub, "Github issue tracker," 2025. [Online]. Available: https://github.com/issues/

[3] Google, "Google code issue tracker," 2025. [Online]. Available: https://code.google.com/archive/

[4] M. Fazzini, M. Prammer, M. d'Amorim, and A. Orso, "Automatically translating bug reports into test cases for mobile apps," in *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2018. New York, NY, USA: Association for Computing Machinery, Jul. 2018, pp. 141–152.

[5] Y. Zhao, T. Yu, T. Su, Y. Liu, W. Zheng, J. Zhang, and W. G.J. Halfond, "ReCDroid: Automatically Reproducing Android Application Crashes from Bug Reports," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. Montréal, QC, Canada: IEEE, May 2019, pp. 128–139.

[6] Y. Zhao, T. Su, Y. Liu, W. Zheng, X. Wu, R. Kavuluru, W. G. J. Halfond, and T. Yu, "ReCDroid+: Automated End-to-End Crash Reproduction from Bug Reports for Android Apps," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 3, pp. 36:1–36:33, Mar. 2022.

[7] Z. Zhang, R. Winn, Y. Zhao, T. Yu, and W. G. Halfond, "Automatically Reproducing Android Bug Reports using Natural Language Processing and Reinforcement Learning," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2023. New York, NY, USA: Association for Computing Machinery, Jul. 2023, pp. 411–422.

[8] Y. Huang, J. Wang, Z. Liu, S. Wang, C. Chen, M. Li, and Q. Wang, "Context-Aware Bug Reproduction for Mobile Apps," in *Proceedings of the 45th International Conference on Software Engineering*, ser. ICSE '23. Melbourne, Victoria, Australia: IEEE Press, Jul. 2023, pp. 2336–2348.

[9] Z. Zhang, F. M. Tawsif, K. Ryu, T. Yu, and W. G. J. Halfond, "Mobile Bug Report Reproduction via Global Search on the App UI Model," *Reproduction Package for "Mobile Bug Report Reproduction via Global Search on the App UI Model"*, vol. 1, no. FSE, pp. 117:2656–117:2676, Jul. 2024.

[10] Z. Zhang, K. Ryu, T. Yu, and W. G. Halfond, "Automated Recognition of Buggy Behaviors from Mobile Bug Reports," *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, pp. FSE100:2240–FSE100:2263, Jun. 2025.

[11] S. Feng and C. Chen, "Prompting Is All You Need: Automated Android Bug Replay with Large Language Models," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. Lisbon Portugal: ACM, Feb. 2024, pp. 1–13.

[12] D. Wang, Y. Zhao, S. Feng, Z. Zhang, W. G. J. Halfond, C. Chen, X. Sun, J. Shi, and T. Yu, "Feedback-Driven Automated Whole Bug Report Reproduction for Android Apps," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2024. New York, NY, USA: Association for Computing Machinery, Sep. 2024, pp. 1048–1060.

[13] OpenAI, "Introducing chatgpt," 2025. [Online]. Available: https://openai.com/index/chatgpt/

[14] ——, "Gpt-4," 2025. [Online]. Available: https://openai.com/index/gpt-4-research/

[15] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, P. Schuh, K. Shi, S. Tsvyashchenko, J. Maynez, A. Rao, P. Barnes, Y. Tay, N. Shazeer, V. Prabhakaran, E. Reif, N. Du, B. Hutchinson, R. Pope, J. Bradbury, J. Austin, M. Isard, G. Gur-Ari, P. Yin, T. Duke, A. Levskaya, S. Ghemawat, S. Dev, H. Michalewski, X. Garcia, V. Misra, K. Robinson, L. Fedus, D. Zhou, D. Ippolito, D. Luan, H. Lim, B. Zoph, A. Spiridonov, R. Sepassi, D. Dohan, S. Agrawal, M. Omernick, A. M. Dai, T. S. Pillai, M. Pellat, A. Lewkowycz, E. Moreira, R. Child, O. Polozov, K. Lee, Z. Zhou, X. Wang, B. Saeta, M. Diaz, O. Firat, M. Catasta, J. Wei, K. Meier-Hellstern, D. Eck, J. Dean, S. Petrov, and N. Fiedel, "PaLM: Scaling Language Modeling with Pathways," Oct. 2022.

[16] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and Efficient Foundation Language Models," Feb. 2023.

[17] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS '22. Red Hook, NY, USA: Curran Associates Inc., 2022.

[18] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS '22. Red Hook, NY, USA: Curran Associates Inc., 2022.

[19] Q. Lyu, S. Havaldar, A. Stein, L. Zhang, D. Rao, E. Wong, M. Apidianaki, and C. Callison-Burch, "Faithful Chain-of-Thought Reasoning," Sep. 2023.

[20] Github, "Amazefilemanager," 2025. [Online]. Available: https://github.com/TeamAmaze/AmazeFileManager

[21] ——, "Ltgdroid," 2025. [Online]. Available: https://github.com/N3onFlux/LTGDroid

[22] T. Wendland, J. Sun, J. Mahmud, S. M. H. Mansur, S. Huang, K. Moran, J. Rubin, and M. Fazzini, "Andror2: A Dataset of Manually-Reproduced Bug Reports for Android apps," in *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. Madrid, Spain: IEEE, May 2021, pp. 600–604.

[23] T. Su, J. Wang, and Z. Su, "Benchmarking automated GUI testing for Android against real-world bugs," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2021. New York, NY, USA: Association for Computing Machinery, Aug. 2021, pp. 119–130.

[24] GitHub, "Github rest api documentation," 2022. [Online]. Available: https://docs.github.com/en/rest

[25] Y. Li, Z. Yang, Y. Guo, and X. Chen, "DroidBot: A lightweight UI-guided test input generator for Android," in *Proceedings of the 39th International Conference on Software Engineering Companion*, ser. ICSE-C '17. Buenos Aires, Argentina: IEEE Press, May 2017, pp. 23–26.

[26] S. Feng and C. Chen, "GIFdroid: Automated replay of visual bug reports for Android apps," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: Association for Computing Machinery, Jul. 2022, pp. 1045–1057.

[27] Y. Huang, J. Wang, Z. Liu, Y. Wang, S. Wang, C. Chen, Y. Hu, and Q. Wang, "CrashTranslator: Automatically Reproducing Mobile Application Crashes Directly from Stack Trace," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, Feb. 2024, pp. 1–13.

[28] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, Dec. 2024. [Online]. Available: https://doi.org/10.1145/3695988

[29] K. Zhang, D. Wang, J. Xia, W. Y. Wang, and L. Li, "Algo: Synthesizing algorithmic programs with llm-generated oracle verifiers," 2023. [Online]. Available: https://arxiv.org/abs/2305.14591

[30] Z. Xue, L. Li, S. Tian, X. Chen, P. Li, L. Chen, T. Jiang, and M. Zhang, "Llm4fin: Fully automating llm-powered test case generation for fintech software acceptance testing," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 1643–1655. [Online]. Available: https://doi.org/10.1145/3650212.3680388

[31] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, "Chatunitest: A framework for llm-based test generation," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, ser. FSE 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 572–576. [Online]. Available: https://doi.org/10.1145/3663529.3663801

[32] D. Ran, H. Wang, Z. Song, M. Wu, Y. Cao, Y. Zhang, W. Yang, and T. Xie, "Guardian: A Runtime Framework for LLM-Based UI Exploration," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2024. New York, NY, USA: Association for Computing Machinery, Sep. 2024, pp. 958–970.

[33] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, "Make llm a testing expert: Bringing human-like interaction to mobile gui testing via functionality-aware decisions," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3597503.3639180